

Chapter 8

The Trouble with Distributed Systems

Partial failures, unreliable networks, lying clocks, and arbitrary pauses — and how to engineer reliability anyway

Source: [claude-skills / ch08-trouble-with-distributed-systems.md](#) · Illustrated · Interview-ready

Martin Kleppmann · Part II: Distributed Data

Table of Contents

1. Core Idea
2. Framework: System Models
3. Safety vs Liveness
4. Unbounded Delays & Timeouts
5. Queueing & Network Variability
6. Packet vs Circuit Switching
7. Clocks: Time-of-Day vs Monotonic
8. Clock Skew, Drift & TrueTime
9. Timestamps Cannot Order Events
10. Process Pauses & Stale Checks
11. Byzantine Faults
12. Mental Models
13. Anti-Patterns
14. Worked Example: Fencing Tokens
15. Problems Each Mechanism Solves
16. Connects To (Cross-Chapter)
17. Key Takeaways
18. Junior SWE Checkpoints

1. Core Idea

A single computer is deterministic: it works or it crashes, never something in between. A distributed system loses that idealization — you get **partial failures** that are non-deterministic.



Unreliable network

Packets lost, duplicated, reordered, or arbitrarily delayed.



Unreliable clocks

Out of sync, jumping forward or backward (NTP, leap seconds).



Unreliable processes

Paused at any point for arbitrary lengths (GC, VM suspend, paging).

Single machine vs distributed partial failure

Single computer

Works ✓ or Crashed ✗
Deterministic, knowable state

Distributed system

Slow? Lost? Crashed? Paused?
Non-deterministic partial failure

A node knows nothing for sure

It can only guess from messages it receives — or fails to receive
Reliability = explicit assumptions (system model) + algorithms proven correct within it

Figure 1.1 — Partial failure replaces the simple "up or down" model.

Reliability must be engineered on top of unreliable components — like TCP over IP. Make assumptions explicit via a **system model**, then design algorithms proven correct within it. A node cannot trust its own judgment; truth is a quorum decision (Ch 9).

2. Framework: System Models

Formalize what an algorithm may assume: **timing** × **node failure**. State your model before claiming any algorithm is correct.

Timing models

Model	Assumption	Reality
Synchronous	Bounded network delay, bounded process pauses, bounded clock error	Unrealistic for practical internet-scale systems
Partially synchronous	Behaves synchronously <i>most</i> of the time, but bounds are occasionally shattered arbitrarily	The realistic model — most real server-side systems
Asynchronous	No timing assumptions at all — not even a clock or timeouts	Very restrictive; few algorithms work here

Node failure models

Model	Behavior	When to use
Crash-stop	Failed node never returns	Simplified analysis; rarely exact in production
Crash-recovery	Nodes crash and may return; stable storage survives, in-memory state lost	Default for real systems — partially synchronous + crash-recovery
Byzantine	Nodes may do anything, including lying and sending corrupted messages	Aerospace, blockchain, mutually untrusting orgs

Timing spectrum: sync → partial → async

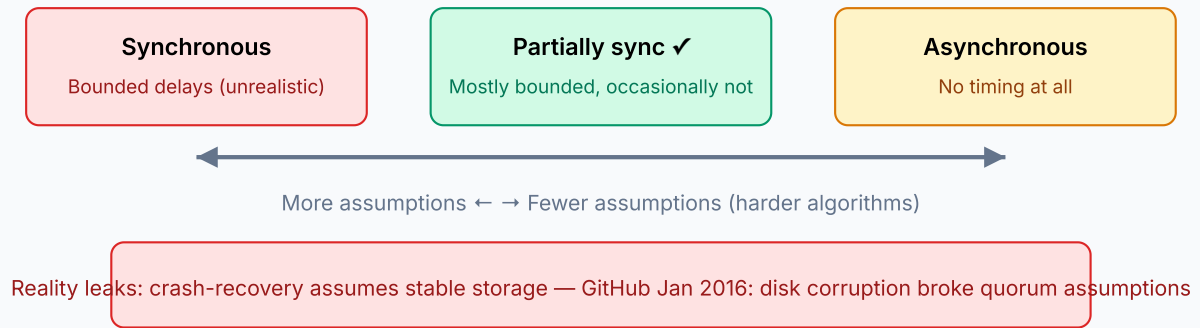


Figure 2.1 — Partially synchronous + crash-recovery is the practical default.

When to use: State your model before claiming correctness. Most real server-side systems: **partially synchronous with crash-recovery faults**. Failure mode: reality leaks — disks corrupt, firmware bugs lose data, silently breaking algorithms that assume nodes remember what they stored.

3. Safety vs Liveness

Define algorithm correctness precisely by splitting properties into two categories.

Property	Meaning	Examples	Requirement
Safety	"Nothing bad happens" — if violated, you can point at the exact moment it broke, and damage cannot be undone	Fencing-token <i>uniqueness, monotonic sequence</i> ; no split-brain writes	Must hold always , even if all nodes crash or network fails entirely
Liveness	"Something good eventually happens" — may not hold now, but hope remains ("eventually" is the giveaway)	<i>Availability</i> ; eventual consistency; "a response will arrive if majority is up"	Conditional — e.g., only if majority is up and network eventually recovers

Safety = never break · Liveness = eventually succeed

SAFETY

"Nothing bad ever happens"

Violation = irreversible damage

Must hold ALWAYS

e.g. token uniqueness, no corrupt writes

LIVENESS

"Something good eventually happens"

Violation = stuck waiting (recoverable)

Conditional on system health

e.g. availability, eventual consistency

Figure 3.1 — Demand hard guarantees where corruption is irreversible; accept best-effort where waiting is tolerable.

Eventual consistency is a liveness property — "eventually" all replicas converge. Safety properties (no lost acknowledged writes, no duplicate primary keys) must never be violated, even during partitions.

4. Unbounded Delays & Timeouts

Asynchronous packet networks give **no delivery-time guarantee**. No response is indistinguishable among many causes.

Possible cause	What actually happened
Request lost	Packet dropped in transit
Request queued	Switch or server backlog
Remote node crashed	Process dead
Node paused (GC)	Process alive but frozen — looks dead from outside
Response lost	Server processed, reply never arrived
Response delayed	Still in flight, will arrive late

Client sends request... then silence. Why?

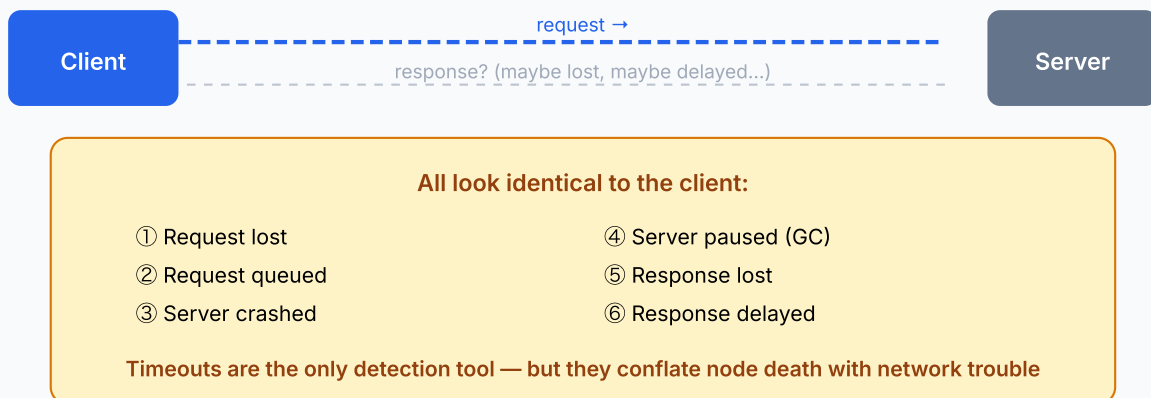


Figure 4.1 — Silence tells you nothing. Timeouts are a guess, not proof.

No "correct" timeout exists. Too-short timeouts falsely declare live nodes dead → load shifts → cascading failure. Measure round-trip distributions and jitter, or adapt continuously (**Phi Accrual failure detector**, used in Akka and Cassandra).

📞 **Calling someone who doesn't pick up.** Are they asleep? Phone dead? In a tunnel? Ignoring you? You can't know — you hang up (timeout) and try again. Same ambiguity, same false-alarm risk.

5. Queueing & Network Variability

Queueing is the source of delay variability. Delays explode near capacity; noisy neighbors in multitenant clouds make them unpredictable.

Switch queues

Congestion under load — packets wait in line.

OS run queues

CPU scheduling, thread contention.

VM scheduling

Hypervisor pauses, CPU steal time.

TCP flow control

Backpressure, retransmission on loss.

Concrete data point	What it means
~12 network faults/month	One medium datacenter — half isolating a machine, half a whole rack
Switch software upgrade	Delayed packets over a minute
NIC driver bug	Drops inbound packets while sending outbound fine — "limping" node worse than dead

🚨 **"Limping" nodes** — a Gigabit NIC dropping to 1 Kb/s from a driver bug is worse than a cleanly failed node. The system keeps routing traffic to a node that technically responds but is effectively useless.

6. Packet vs Circuit Switching

Model	How it works	Delay	Trade-off
Circuit switching (telephone)	Reserves fixed bandwidth end-to-end for the call duration	Bounded delay	Low utilization — bandwidth idle when not speaking
Packet switching (Ethernet/IP)	Bursty traffic shares links; packets queue under congestion	Variable delay	High utilization at cost of queueing — a cost/utilization trade-off, not a law of nature

Circuit = reserved lane · Packet = shared highway with traffic jams

Circuit: dedicated 64 Kb/s

Predictable delay, wasted capacity

Packet: shared link, burst-friendly

High utilization, queueing delays

Bounded-delay networks are possible — but at utilization/cost most internet services won't pay
If you can stay on a single machine, do — distribution is for fault tolerance and latency, not fashion

Figure 6.1 — Variable delay is an engineering trade-off, not inevitability.

7. Clocks: Time-of-Day vs Monotonic

Clock type	API	Behavior	Use for
Time-of-day (wall-clock)	<code>clock_gettime(CLOCK_REALTIME)</code> <code>System.currentTimeMillis()</code>	NTP-synced but can be forcibly reset — jumps backwards	Human-readable timestamps, log correlation
Monotonic	<code>CLOCK_MONOTONIC</code> <code>System.nanoTime()</code>	Always moves forward; absolute values meaningless across machines	Timeouts, durations, latency measurement

Wall clock can jump · Monotonic never goes backward



Never use time-of-day for elapsed time or event ordering across nodes

Figure 7.1 — Monotonic for durations; wall clock for human timestamps only.

Never compare a lease expiry set by a remote clock against local time-of-day — clock skew silently breaks the lease protocol.

8. Clock Skew, Drift & TrueTime

Phenomenon	Data	Impact
Quartz drift	Google assumes 200 ppm → 6 ms drift per 30 s sync interval; 17 s/day	Clocks diverge between NTP syncs
NTP over internet	~35 ms minimum error; spikes to ~1 s under congestion	Cross-datacenter ordering unreliable
Leap seconds	59- or 61-second minutes	Have crashed major systems — mitigate by <i>smearing</i> over a day

A clock reading is really a confidence interval, not a point. Spanner's **TrueTime** returns [earliest, latest] and waits out the interval before committing so transaction timestamps reflect causality. GPS/atomic clocks per datacenter keep uncertainty ~7 ms.

Spanner TrueTime: clock = interval, not point



Commit waits until `TT.now().earliest > commit timestamp`

Connects to Ch 7 snapshot isolation's monotonic transaction IDs across datacenters

Snowflake IDs (41-bit timestamp) must own up to what happens when clocks drift

Figure 8.1 — Treat wall-clock readings as intervals; wait out uncertainty for causal ordering.

9. Timestamps Cannot Order Events

Last-write-wins (LWW) in Cassandra/Riak with time-of-day timestamps silently drops causally-later writes when the later writer's clock lags — data vanishes with no error.

LWW + clock skew = silent data loss

Node A writes at T=100 (local clock)

Causally first — sets color=blue

Node B writes at T=95 (clock lagging!)

Causally later — sets color=red

LWW picks T=100 (blue) — red write silently discarded

No error, no conflict — data just vanishes

Fix: logical clocks (counters tracking happens-before) for ordering, not physical clocks

Figure 9.1 — Physical timestamps ≠ causal order. Ch 5 LWW conflicts bite here.

Use **logical clocks** (Lamport timestamps, version vectors from Ch 5) for ordering. Physical clocks for human display only. Connects to Ch 5 replication LWW and Ch 7 transaction IDs.

10. Process Pauses & Stale Checks

A thread can be preempted at any point for any duration and resume without noticing — so a lease-validity check made before processing may be stale by the time the write executes.

Stop-the-world GC

Minutes have been observed — blows p99 SLA.

VM suspension

Live migration, hypervisor scheduling.

Disk I/O / paging

Synchronous I/O, page faults, thrashing.

SIGSTOP / laptop lid

Process frozen externally, resumes unaware.

GC pause makes client-side lease check useless

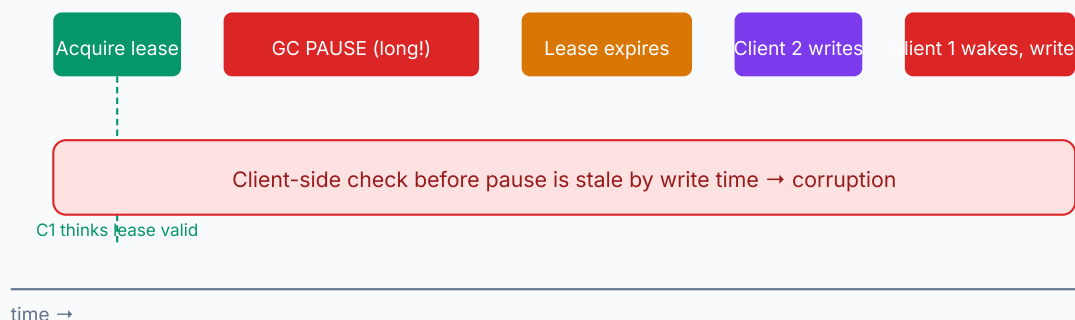


Figure 10.1 — The check itself can be stale. Protect resources server-side (fencing tokens).

 **Distributed systems = multithreading without shared memory.** Arbitrary pauses and interleavings, but none of the tools (mutexes, atomics) translate — only messages over an unreliable network. Assume execution can stop mid-function while the rest of the world moves on.

11. Byzantine Faults

Nodes that "lie" — send arbitrary corrupted or malicious messages (**Byzantine Generals Problem**, Lamport).

Context	Why Byzantine matters
Aerospace	Radiation flips memory bits
Blockchain / multi-org	Mutually untrusting participants
Most datacenters	Rightfully assume non-Byzantine — you control all nodes

BFT cost

Need $>2/3$ supermajority of correct nodes.
Expensive protocols.

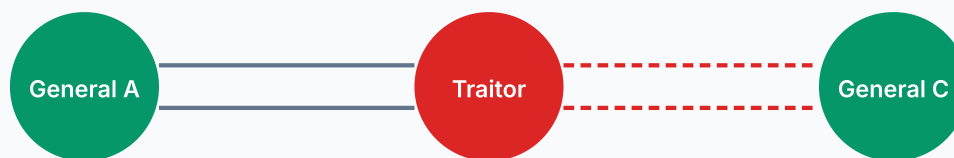
Same software

Attacker who compromises one node can likely compromise all — auth/firewalls are the real defense.

Weak lying

Still guard: checksums, input sanitization, NTP outlier rejection.

Byzantine Generals: loyal vs traitor nodes



Traitor sends conflicting messages → agreement impossible without $>2/3$ honest majority

Fencing stops error, not deliberate forgery — that's Byzantine territory

Figure 11.1 — Most datacenter systems assume crash-recovery, not Byzantine.

12. Mental Models

1. Truth = majority

A node cannot trust its own judgment — semi-disconnected, or waking from 1-minute GC not knowing it was declared dead. Decisions (including "is X dead?") belong to a **quorum**. One majority only; outvoted nodes must step down.

2. Multithreading without shared memory

Arbitrary pauses and interleavings, but no mutexes/atomics — only messages over unreliable network. Execution can stop mid-function while the world moves on.

3. Reliable from unreliable (up to a limit)

TCP over IP, error-correcting codes. Higher layer removes some fault classes (loss, duplication, reordering) but not others (delay). Know which faults your abstraction absorbs.

4. Cloud vs supercomputer

HPC: partial failure → total failure, restart from checkpoint. Internet services: tolerate failed nodes while serving. At scale, "something is always broken" — giving up is not a strategy.

Node declared dead by quorum must step down — even if it feels alive

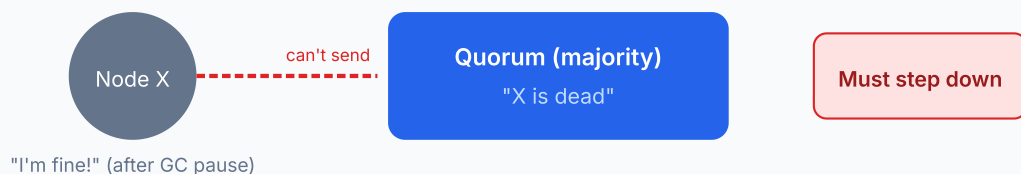


Figure 12.1 — Self-assessment is worthless; quorum decides truth.

13. Anti-Patterns

Anti-pattern	Why it fails
Comparing lease expiry (remote clock) against local time-of-day	Clock skew silently breaks lease protocol
Trusting client's belief it holds lock/lease without server-side verification	Split-brain writers after GC pause or partition
LWW conflict resolution on time-of-day timestamps	Causally later writes silently dropped — no error surfaced
Timeouts as constants picked by intuition	False-positive death declarations → cascading failure. Measure RTT distributions or use Phi Accrual
Assuming node is working or dead (binary)	"Limping" nodes (1 Kb/s NIC) worse than cleanly failed — system keeps routing to them
Relying on synchronized clocks without monitoring offsets	Drifting clock causes silent data loss, not crash — evict nodes whose clocks drift too far
Leaving network-fault handling undefined/untested	Clusters deadlocked permanently; netsplit deleted all data (Elasticsearch, Redis Sentinel). Inject faults deliberately (chaos monkey)

14. Worked Example: Fencing Tokens

Scenario: a file in a storage service must be written by one client at a time, enforced by leases from a lock service (**HBase actually had this bug**).

The bug: lease + GC pause

1. Client 1 acquires lease for file F
2. Client 1 hits stop-the-world GC pause longer than lease TTL
3. Lease expires; Client 2 acquires lease and writes to F
4. Client 1 wakes, still believes lease valid (nothing told the paused thread)
5. Client 1 writes too → writes clash and corrupt F

Client-side lease checking CANNOT fix this:

```
check_lease() // passes – lease was valid when checked
[GC PAUSE – lease expires, Client 2 writes]
write_to_file() // stale check – corruption
```

The fix: server-side fencing tokens

Lock service returns a fencing token with every grant –
a number that increases monotonically (ZooKeeper zxid or cversion).

Every write must carry its token.

Storage service checks and rejects any write whose token $\leq$ last processed.

Timeline:

```
Client 1 gets token 33, then GC pauses
Client 2 gets token 34, writes (storage records max=34) ✓
Client 1 wakes, writes with token 33 → REJECTED (33 < 34) ✓
```

Required properties:

Safety: uniqueness + monotonic sequence (must NEVER break)

Liveness: availability (lock service keeps granting tokens)

Fencing tokens: storage rejects stale writes

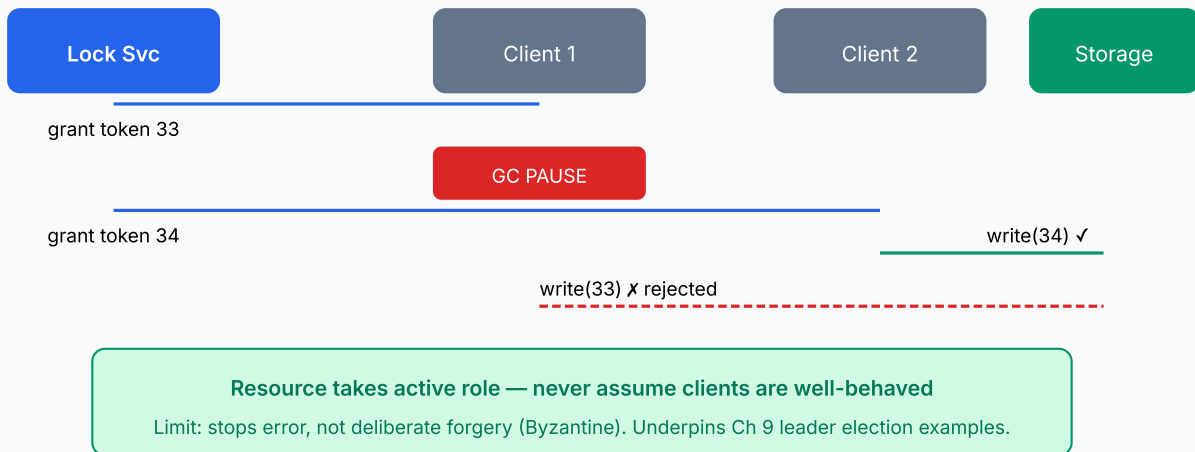


Figure 14.1 — Monotonic fencing tokens make stale writes safe to reject server-side.

Key insight: The resource (storage) takes an active role in enforcement. Client-side lease checks are always stale-able. Fencing provides *safety* (uniqueness + monotonicity); lock service availability is *liveness*.

15. Problems Each Mechanism Solves

Mechanism	Problem it solves	Example
Named clock-sync assumption ("NTP-synchronized clocks")	ID generator must produce roughly time-ordered IDs without coordination — and must own up to what happens when clocks drift	Unique-ID generator (Snowflake's 41-bit timestamp)
GC-pause / tail-latency awareness	Stop-the-world pause at the wrong moment blows the p99 SLA	Stock exchange (explicit p99-latency + GC-pause callout)
Heartbeat + tuned timeout for liveness	Distinguish "offline" from "just slow" without false-positive churn	Chat system presence; metrics monitoring pull-model scrape as health check
Short timeouts + defense against indistinguishable-slow-vs-dead	Crawler must not let one unresponsive host stall the whole frontier	Web crawler
Accepting retry ≠ knowing failure	Third-party API call outcome genuinely unknown until it resolves	Notification system (APNS/FCM/SendGrid retries)
Fencing tokens on the resource, not client-side lease checks	Paused leader/lock-holder waking late must not corrupt state	Underpins every leader-election use in Ch 9 (S3 placement, digital wallet, stock exchange)

16. Connects To (Cross-Chapter)

Chapter	Connection	Interview soundbite
Ch 5 Replication	Leader failover, replication lag, and LWW conflicts are where these faults first bite	"Never order events by wall clock — LWW silently drops writes when clocks skew."
Ch 7 Transactions	Snapshot isolation's monotonic transaction IDs are what Spanner's TrueTime approximates across datacenters	"TrueTime waits out clock uncertainty so commit timestamps respect causality."
Ch 9 Consistency & Consensus	The payoff — quorums, consensus, and total ordering provide guarantees within the partially-synchronous/crash-recovery model this chapter motivates	"Truth is a quorum decision. Ch 9 gives you the algorithms."

17. Key Takeaways

1. **Partial failure is the defining property** of distributed systems: operations non-deterministically fail, go slow, or vanish — and you may never learn whether they succeeded.
2. **No response tells you nothing about why**; timeouts are the only failure detector and they conflate node death with network trouble.
3. **Never use time-of-day clocks** for elapsed time or event ordering; monotonic clocks for durations, logical clocks for ordering, and treat any wall-clock reading as a confidence interval.
4. **Any thread can pause arbitrarily long** at any point (GC, VM suspend, paging) and resume unaware — protect resources server-side with fencing tokens, not client-side lease checks.
5. **A node's self-assessment is worthless**; truth is a quorum decision, and outvoted nodes must step down.
6. **Design against an explicit system model** (usually partially synchronous + crash-recovery); require safety always, liveness conditionally.
7. **If you can stay on a single machine, do** — distribution is justified by fault tolerance and latency (data near users), not fashion; bounded-delay networks require utilization/cost most systems won't pay.

18. Junior SWE Checkpoints

Q1: Why can't you tell why a request got no response?

Request lost, queued, server crashed, server paused (GC), response lost, or response delayed — all look identical (silence). Timeouts are a guess that conflates node failure with network trouble.

Q2: Time-of-day vs monotonic clock — when to use each?

Time-of-day (wall clock): human timestamps, log correlation — but can jump backward (NTP).

Monotonic (`nanoTime`): timeouts and durations — always forward, but meaningless across machines. Never use wall clock for elapsed time or cross-node ordering.

Q3: Why does LWW with timestamps silently lose data?

Later writer's clock may lag earlier writer's. LWW picks higher timestamp, discarding the causally-later write — no error, no conflict surfaced. Fix: logical clocks (version vectors, Lamport timestamps).

Q4: Client checks lease valid, then writes. Why can this still corrupt data?

GC pause (or VM suspend) between check and write. Lease expires; another client acquires and writes. First client wakes, check was stale. Fix: fencing tokens checked by the *storage service*, not the client.

Q5: Safety vs liveness — give an example of each.

Safety ("nothing bad"): fencing token uniqueness — must never break, even if all nodes crash. Liveness ("eventually good"): availability, eventual consistency — conditional on majority up and network recovery.

Q6: Why do most datacenters assume crash-recovery, not Byzantine?

You control all nodes; same software everywhere — compromise one, likely compromise all. BFT needs $>2/3$ honest nodes and expensive protocols. Real defense: auth, access control, firewalls. Still guard weak lying with checksums and NTP outlier rejection.

Q7: How does this chapter connect to Ch 9 consensus?

Ch 8 motivates the problem (partial failure, unreliable components) and the system model (partially sync + crash-recovery). Ch 9 delivers the algorithms — quorums, Raft/Paxos, leader election — that provide safety and liveness guarantees within that model. Fencing tokens underpin Ch 9's leader-election examples.

Glossary

Term	Definition
Partial failure	Some components fail while others continue — non-deterministic, unlike single-machine crash
System model	Formal assumptions about timing and node failure types an algorithm relies on
Partially synchronous	Mostly bounded delays, occasionally shattered — realistic model for datacenters
Crash-recovery	Nodes crash and may return; stable storage survives, in-memory state lost
Safety property	"Nothing bad happens" — must hold always; violation is irreversible
Liveness property	"Something good eventually happens" — conditional on system health
Phi Accrual failure detector	Adaptive timeout based on RTT distribution (Akka, Cassandra)
Monotonic clock	Always moves forward — safe for timeouts/durations, not cross-node ordering
TrueTime	Spanner's clock API returning uncertainty interval [earliest, latest]
Logical clock	Counter tracking happens-before — safe for event ordering across nodes
Fencing token	Monotonically increasing number from lock service; storage rejects stale tokens
Byzantine fault	Node sends arbitrary/corrupted/malicious messages — needs $>2/3$ honest majority
Limping node	Technically alive but effectively broken (e.g., NIC at 1 Kb/s) — worse than dead

